

Principios SOLID

Los principios **SOLID** son cinco reglas de diseño orientado a objetos que ayudan a crear software más mantenible, flexible y fácil de extender. Aunque el artículo usa Java, los conceptos aplican a cualquier lenguaje OO. El acrónimo SOLID significa:

- Single Responsibility Principle (Principio de responsabilidad **única**)
- Open/Closed Principle (Principio **abierto /cerrado**)
- Liskov Substitution Principle (Principio de sustitución de **L' iskov**)
- Interface Segregation Principle (Principio de segregación de **interfaz**)
- Dependency Inversion Principle (Principio de inversión de **dependencia**)

1. S – Single Responsibility Principle (SRP) (Principio de responsabilidad única)

Una clase debe tener una sola razón para cambiar.

Cada clase debe encargarse de una sola cosa. Si hace demasiadas tareas, se vuelve difícil de mantener.

Ejemplo: Una clase Factura no debería:

- calcular el total,
- imprimir la factura,
- guardarla en la base de datos.

Lo correcto sería dividirlo en:

- FacturaCalcular
- FacturaImprimir
- FacturaRepositorio

Así, si cambia la forma de imprimir, no tocas la lógica de cálculo.

2. O – Open/Closed Principle (OCP) (Principio abierto /cerrado)

Las clases deben estar abiertas a extensión, pero cerradas a modificación.

No modificar código existente para agregar nuevas funcionalidades; extenderlo.

Ejemplo: Si tienes una clase que calcula descuentos, no deberías modificarla cada vez que aparece un nuevo tipo de descuento. En su lugar, defines una interfaz Descuento y creas nuevas implementaciones.

Esto evita romper lo que ya funciona.

3. L – Liskov Substitution Principle (LSP) (Principio de sustitución de L'iskov)

Las clases hijas deben poder sustituir a sus padres sin romper el programa.

Si una subclase cambia el comportamiento esperado, entonces no es una verdadera subclase.

Ejemplo: Rectángulo y cuadrado. Un cuadrado no puede comportarse como un rectángulo sin romper expectativas (altura $\neq$ anchura). Por eso, no deberían tener relación de herencia.

4. I – Interface Segregation Principle (ISP) (Principio de segregación de interfaz)

Las interfaces deben ser pequeñas y específicas, no gigantes y genéricas.

No fuerces a una clase a implementar métodos que no necesita.

Ejemplo: Una interfaz Trabajador con métodos:

- trabajar()
- comer()

No tiene sentido para un robot. Mejor dividir en:

- Trabajable
- Comestible

5. D – Dependency Inversion Principle (DIP) (Principio de inversión de dependencia)

Depende de abstracciones, no de implementaciones concretas.

Las clases de alto nivel no deben depender de clases de bajo nivel, sino de interfaces.

Ejemplo: En vez de que ServicioDeNotificación dependa directamente de RemitenteCorreo, debería depender de una interfaz RemitenteMensaje.

Así puedes cambiar a SMS, WhatsApp, Push... sin tocar el servicio principal.

¿Por qué importan los SOLID?

Aplicarlos te da:

- código más limpio,
- menos bugs,
- facilidad para añadir nuevas funcionalidades,
- mejor testabilidad,
- menor acoplamiento.

En proyectos grandes, marcan la diferencia entre un sistema manejable y un infierno técnico.

Ejemplos en java

1. S – Single Responsibility Principle (SRP) (Principio de responsabilidad única)

Una clase debe tener una sola razón para cambiar.

Ejemplo: Gestión de facturas

Mala práctica (una clase hace demasiado)

```
public class Factura {  
    public void calcularTotal() { /* ... */ }  
    public void guardarEnBD() { /* ... */ }  
    public void enviarPorEmail() { /* ... */ }  
}
```

Buena práctica (cada clase tiene una responsabilidad)

```
public class Factura {  
    public void calcularTotal() { /* ... */ }  
}  
  
public class FacturaRepositorio {  
    public void guardar(Factura factura) { /* ... */ }  
}  
  
public class ServicioEmailFactura {  
    public void enviar(Factura factura) { /* ... */ }  
}
```

2. O – Open/Closed Principle (OCP) (Principio abierto /cerrado)

Las clases deben estar abiertas a extensión, pero cerradas a modificación.

Ejemplo: Sistema de descuentos

Mala práctica: cada nuevo descuento obliga a modificar la clase

```
public class ServicioDescuento {  
    public double aplicar(String tipo, double precio) {  
        if (tipo.equals("NAVIDAD")) return precio * 0.9;  
        if (tipo.equals("BLACK_FRIDAY")) return precio * 0.8;  
        return precio;  
    }  
}
```

Buena práctica: se extiende sin tocar el código existente

```
public interface Descuento {  
    double aplicar(double precio);  
}  
  
public class DescuentoNavidad implements Descuento {  
    public double aplicar(double precio) { return precio * 0.9; }  
}  
  
public class DescuentoBlackFriday implements Descuento {  
    public double aplicar(double precio) { return precio * 0.8; }  
}  
  
public class ServicioDescuento {  
    public double aplicar(Descuento descuento, double precio) {  
        return descuento.aplicar(precio);  
    }  
}
```

3. L – Liskov Substitution Principle (LSP) (Principio de sustitución de L' iskov)

Las subclases deben poder reemplazar a sus clases padre sin romper el programa.

Ejemplo: Vehículos

Mala práctica: una subclase rompe el comportamiento esperado

```
public class Vehiculo {  
    public void acelerar() { System.out.println("Acelerando"); }  
}  
  
public class VehiculoRoto extends Vehiculo {  
    @Override  
    public void acelerar() {  
        throw new UnsupportedOperationException("No puedo acelerar");  
    }  
}
```

Buena práctica: todas las subclases cumplen el contrato

```
public interface Vehiculo {  
    void acelerar();  
}  
  
public class Coche implements Vehiculo {  
    public void acelerar() { System.out.println("Coche acelerando"); }  
}  
  
public class Moto implements Vehiculo {  
    public void acelerar() { System.out.println("Moto acelerando"); }  
}
```

4. I – Interface Segregation Principle (ISP) (Principio de segregación de interfaz)

No obligues a implementar métodos que no se necesitan.

Ejemplo: Trabajadores

Mala práctica: una interfaz demasiado grande

```
public interface Trabajador {  
    void trabajar();  
    void comer();  
}
```

Un robot no come, pero se ve obligado a implementar comer().

Buena práctica: interfaces pequeñas y específicas

```
public interface Trabajable {  
    void trabajar();  
}
```

```
public interface Comible {  
    void comer();  
}
```

```
public class Humano implements Trabajable, Comible {  
    public void trabajar() { /* ... */ }  
    public void comer() { /* ... */ }  
}
```

```
public class Robot implements Trabajable {  
    public void trabajar() { /* ... */ }  
}
```

5. D – Dependency Inversion Principle (DIP) (Principio de inversión de dependencia)

Depende de abstracciones, no de implementaciones concretas.

Ejemplo: Servicio de notificaciones

Mala práctica: la clase depende directamente de EmailSender

```
public class ServicioNotificacion {  
    private EmailSender emailSender = new EmailSender();  
  
    public void enviar(String mensaje) {  
        emailSender.enviarEmail(mensaje);  
    }  
}
```

Buena práctica: se invierte la dependencia

```
public interface EnviadorMensaje {  
    void enviar(String mensaje);  
}  
  
public class EmailSender implements EnviadorMensaje {  
    public void enviar(String mensaje) { /* ... */ }  
}  
  
public class SmsSender implements EnviadorMensaje {  
    public void enviar(String mensaje) { /* ... */ }  
}  
  
public class ServicioNotificacion {  
    private final EnviadorMensaje enviador;  
  
    public ServicioNotificacion(EnviadorMensaje enviador) {
```



```
        this.enviador = enviador;
    }

    public void enviar(String mensaje) {
        enviador.enviar(mensaje);
    }
}
```